Trabajo Práctico 2 – TDA Diccionario e Implementaciones Dinámicas



Profesor: Coedo Maximiliano Gonzalo

Alumnos: Vergara Alejo, Blazquez Catalina y Fiore Micaela

ÍNDICE

Parte 0 — Preguntas teóricas 3
Parte 1 — Especificación TDA Diccionario 3
Parte 2.1 — Complejidad teórica 4
3.1 — Medición empírica 5
3.2 — Conclusión 6
Parte 4 — Complejidad temporal de los métodos de utilización 6

RESPUESTAS:

Parte 0 — Preguntas teóricas

1. El **Conjunto** almacena elementos sin repetir, mientras que el **Diccionario** almacena pares **clave–valor**, donde cada clave identifica un valor asociado. El Diccionario necesita una operación como existeClave antes de definir/agregar una clave porque **no puede haber dos elementos con la misma clave**; primero hay que verificar si ya existe.
2. Un Nodo es la estructura que representa un elemento en una implementación dinámica y permite enlazarlo con otros. Como mínimo debe guardar **el dato/elemento** y una **referencia o enlace al siguiente nodo**.
3. En una implementación dinámica, insertar al principio es **O(1)** porque solamente hay que crear/enlazar un nodo y actualizar la referencia al primero. En una implementación estática con arreglo, es **O(n)** porque hay que **desplazar los elementos existentes** una posición para hacer lugar.
4. **Memoria — dinámica vs. estática:**
   Ventaja: la implementación dinámica utiliza memoria según la cantidad de elementos necesarios, evitando reservar de antemano un tamaño fijo.
   Desventaja: cada nodo necesita memoria adicional para almacenar sus referencias/enlaces.
5. Big O describe cómo crece el costo de una operación a medida que aumenta el tamaño de la entrada, pero no representa exactamente el tiempo real. Dos implementaciones con el mismo Big O pueden diferir por factores como **constantes, acceso a memoria, cantidad de operaciones internas o características del hardware**.
6. La medición empírica permite conocer **cuánto tarda realmente** una implementación en un entorno concreto. El análisis Big O permite saber **cómo escala** su costo cuando aumenta el tamaño de la entrada, independientemente del hardware. Por eso se complementan: medir tiempos no reemplaza al análisis teórico porque los resultados empíricos pueden variar según la máquina, JVM, carga del sistema, etc.

Parte 1 — Especificación TDA Diccionario

TDA Diccionario

Dominio: D = Colección de pares clave-valor, donde cada clave es única y puede estar asociada, como máximo, a un valor.

Operaciones:

crear() -> Diccionario
post: devuelve un Diccionario vacío

definir(d: Diccionario, clave, valor) -> Diccionario
pre: d es un Diccionario válido y clave es válida
post: si clave no existía en d, se agrega el par (clave, valor); si ya existía, se actualiza su valor por valor

obtener(d: Diccionario, clave) -> valor
pre: d no está vacío y existeClave(d, clave) = true
post: devuelve el valor asociado a clave, sin modificar d

eliminar(d: Diccionario, clave) -> Diccionario
pre: existeClave(d, clave) = true
post: se elimina de d el par asociado a clave

existeClave(d: Diccionario, clave) -> boolean
post: devuelve true si clave pertenece a d y false en caso contrario, sin modificar d

esVacio(d: Diccionario) -> boolean
post: devuelve true si d no tiene claves, false en caso contrario

cantidadClaves(d: Diccionario) -> entero
post: devuelve la cantidad de claves que tiene d

claves(d: Diccionario) -> Lista
post: devuelve una Lista con todas las claves de d, sin repetirse y sin ningún orden particular; d no se modifica

Parte 2.1 — Complejidad teórica

Operación	Complejidad	Justificación
crear()	**O(1)**	Solo inicializa la cabeza en null y el contador de cantidad en 0, por lo que no depende de la cantidad de elementos.
definir(d, clave, valor)	**O(n)**	Se debe recorrer la cadena para verificar si la clave ya existe. En el peor caso se recorren los n nodos. Si la clave no existe, la inserción al principio es O(1), pero la

		búsqueda determina la complejidad total.
obtener(d, clave)	**O(n)**	Se realiza una búsqueda lineal nodo por nodo para encontrar la clave. En el peor caso, la clave está en el último nodo o no existe.
eliminar(d, clave)	**O(n)**	Se debe recorrer la cadena para encontrar el nodo correspondiente a la clave y poder eliminarlo. En el peor caso se recorren los n nodos. El desenganche del nodo es O(1), pero la búsqueda domina la operación.
existeClave(d, clave)	**O(n)**	Se realiza una búsqueda lineal. En el peor caso, la clave no existe y es necesario recorrer todos los nodos.
esVacio(d)	**O(1)**	Solo se verifica si la cabeza del Diccionario es null. No es necesario recorrer la cadena.
cantidadClaves(d)	**O(1)**	Se mantiene un contador de la cantidad de claves, por lo que solo es necesario devolver su valor.
claves(d)	**O(n)**	Se debe recorrer la cadena completa para obtener las n claves. Cada clave se agrega a la lista en O(1), por lo que el recorrido completo determina una complejidad O(n).

3.1 — Medición empírica

Se realizó una medición empírica del tiempo de ejecución de las operaciones definir() y obtener() para las implementaciones estática y dinámica del diccionario.

Para cada tamaño n (1.000, 10.000 y 100.000 claves), se utilizó la técnica de System.nanoTime(), realizando un *warmup* previo para obtener() y varias repeticiones de cada medición. Para reducir el efecto de variaciones externas, se tomó el mejor tiempo obtenido entre las repeticiones.

En definir() se midió la inserción de una clave nueva, asegurando que la clave utilizada no hubiera sido definida previamente. Para obtener() se seleccionó una clave ubicada en la posición que obliga a recorrer toda la estructura, buscando representar el peor caso de ambas implementaciones.

Los resultados obtenidos fueron:

Cantidad de claves (n)	Definir Estática	Definir Dinámica	Obtener Estática	Obtener Dinámica
1.000	51.400 ns	48.200 ns	39.700 ns	33.400 ns
10.000	124.200 ns	144.100 ns	184.300 ns	130.200 ns
100.000	1.158.200 ns	1.068.000 ns	562.600 ns	838.500 ns

Los tiempos corresponden a ejecuciones reales realizadas sobre el programa implementado y se expresan en nanosegundos (ns).

3.2 — Conclusión

Los resultados obtenidos son compatibles con la complejidad teórica analizada en el punto anterior. Tanto definir() como obtener() presentan un comportamiento **O(n)** en las dos implementaciones cuando se considera el peor caso, ya que es necesario recorrer una parte importante de la estructura para encontrar una clave o determinar dónde insertar una nueva.

Al aumentar la cantidad de claves de 1.000 a 100.000, los tiempos medidos tienden a aumentar, lo que coincide con el comportamiento esperado para operaciones lineales. Sin embargo, los tiempos no aumentan exactamente en la misma proporción que n, debido a factores propios de la ejecución de la máquina virtual de Java, como el compilador JIT, la caché y otras condiciones del sistema.

Aunque ambas implementaciones tienen la misma complejidad asintótica, pueden presentar tiempos diferentes. Esto se debe a las características de cada estructura. La implementación estática utiliza arreglos, cuyos elementos se encuentran almacenados de forma contigua en memoria, favoreciendo la **localidad de referencia** y el acceso eficiente a posiciones consecutivas. En cambio, la implementación dinámica utiliza nodos enlazados, donde cada nodo contiene referencias y puede encontrarse en distintas posiciones de memoria.

Además, la estructura dinámica tiene un mayor **overhead de memoria**, debido a que cada nodo almacena información adicional, como la referencia al siguiente nodo. Por lo tanto, dos estructuras con la misma complejidad O(n) pueden presentar distintos tiempos de ejecución en la práctica.

En conclusión, las mediciones empíricas siguen la tendencia esperada por el análisis teórico.

Parte 4 — Complejidad temporal de los métodos de utilización

Método	Complejidad	Justificación
combinarDiccionarios(d1, d2)	**O(n²)**	Recorre las claves de d1 y d2. Para cada clave realiza claves.obtener(i), que es O(n) porque la lista enlazada debe recorrer los nodos hasta la posición i. Además, realiza obtener() y definir() sobre diccionarios, que son O(n) en el peor caso. Estas operaciones se repiten para O(n) claves, dando un costo total de $O(n^2)$.
invertir(d)	**O(n²)**	Recorre las n claves del diccionario. En cada iteración, claves.obtener(i) es O(n), mientras que d.obtener(clave) y resultado.definir(...) son O(n) en el peor caso. Al repetirse estas operaciones n veces, el costo total es $O(n^2)$.
contarValoresMayoresA(d, umbral)	**O(n²)**	Recorre las n claves. Para cada posición, claves.obtener(i) puede costar O(n) debido a que Lista es enlazada. También se realiza d.obtener(clave), que es O(n) en el peor caso. Al repetirse para n claves, el costo total es $O(n^2)$.
clavesOrdenadas(d)	**O(n²)**	Primero copia las claves al arreglo aux. Cada claves.obtener(i) puede costar O(n), por lo que esta etapa es $O(n^2)$. Luego utiliza ordenamiento por selección, cuyos dos ciclos anidados tienen costo $O(n^2)$. Finalmente, agregarAlFinal() es O(1) porque Lista mantiene una referencia al último nodo, por lo que construir la lista cuesta O(n). El término dominante es $O(n^2)$.